

Rapport de projet

Automatisation des tests avec Cypress

Page Object Model – Allure – Docker – Jenkins

Travail réalisé par :

BEN AMAR YEDIR

Ikram Mabizari

Framework utilisé : Cypress

Architecture : Page Object Model (POM)

Projet : Automatisation des tests End-to-End

Table des matières

1	Automatisation des tests avec Cypress et intégration continue	2
1.1	Introduction	2
1.2	Choix du framework Cypress	2
1.3	Pourquoi automatiser les tests ?	3
1.4	Utilisation du Page Object Model	3
1.5	Organisation des Page Objects	4
1.6	Méthodes utilisées dans le POM	4
1.7	Scénario fonctionnel automatisé	5
1.8	Utilisation de Faker	6
1.9	Synchronisation avec les requêtes réseau	6
1.10	Configuration de Cypress	6
1.11	Problèmes rencontrés et solutions	7
1.12	Gestion du changement de domaine avec cy.origin()	7
1.13	Génération des rapports avec Allure	8
1.14	Résultats de l'automatisation	8
1.14.1	Figure 1 – Rapport Allure : vue générale	9
1.14.2	Figure 2 – Exécution Cypress et redirection vers PayZen	9
1.14.3	Figure 3 – Rapport Allure : détail de la suite	10
1.14.4	Figure 4 – Preuve de réussite du parcours End-to-End	11
1.14.5	Analyse globale des résultats	11
1.15	Intégration continue avec Jenkins	11
1.16	Utilisation de Docker	12
1.16.1	Configuration de Docker avec Jenkins sous Windows	12
1.16.2	Architecture de communication Docker et Jenkins	15
1.16.3	Problème de communication entre Jenkins et Docker	16
1.16.4	Intégration de Docker dans le pipeline Jenkins	17
1.16.5	Figure 5 – Résultat de l'exécution du pipeline Jenkins	17
1.17	Architecture globale de la solution	18
1.18	Bilan du projet	18
1.19	Conclusion	19

Automatisation des tests avec Cypress et intégration continue

1.1 Introduction

Dans le cadre de ce projet, nous avons mis en place une solution d'automatisation des tests fonctionnels d'une application web en utilisant le framework Cypress.

L'objectif principal est d'automatiser un parcours utilisateur complet sur le site Glorria, depuis l'accès à la page d'accueil jusqu'à l'accès à la plateforme de paiement.

Le scénario automatisé couvre plusieurs étapes importantes du parcours utilisateur : l'accès au compte, la création d'un compte, le renseignement des informations personnelles, la sélection d'un produit, la saisie des informations nécessaires à la commande et enfin l'accès à la plateforme de paiement sécurisée.

Afin d'obtenir une architecture claire et maintenable, nous avons utilisé le modèle de conception **Page Object Model (POM)**.

Nous avons également utilisé plusieurs outils complémentaires, notamment Faker pour la génération de données de test, Allure pour la génération des rapports, Jenkins pour l'intégration continue et Docker pour disposer d'un environnement d'exécution reproductible.

1.2 Choix du framework Cypress

Cypress est le framework choisi pour automatiser les tests End-to-End de l'application web.

Il permet de contrôler directement le navigateur et de reproduire les actions réalisées par un utilisateur réel.

Grâce à Cypress, il est possible d'effectuer différentes opérations sur l'application :

- ouvrir une page web ;
- cliquer sur des liens et des boutons ;
- remplir des champs de formulaire ;

- sélectionner des cases à cocher ;
- vérifier les URLs ;
- vérifier les résultats obtenus ;
- intercepter les requêtes réseau ;
- gérer les changements de domaine ;
- visualiser l'exécution du scénario dans le Test Runner.

L'un des avantages importants de Cypress est la possibilité de visualiser les différentes commandes exécutées pendant le test. Cela facilite la compréhension du scénario et le diagnostic des erreurs.

Cypress dispose également d'un mécanisme d'attente automatique des éléments. Cela permet d'éviter dans de nombreux cas l'utilisation d'attentes fixes.

1.3 Pourquoi automatiser les tests ?

L'automatisation permet de réduire le temps nécessaire à l'exécution des tests répétitifs.

Dans un test manuel, le testeur doit effectuer chaque action individuellement et vérifier manuellement le résultat.

Avec l'automatisation, les différentes étapes sont exécutées automatiquement et les assertions permettent de vérifier les résultats attendus.

Les principaux avantages sont :

- gain de temps ;
- réduction des erreurs humaines ;
- possibilité de répéter les tests facilement ;
- détection rapide des régressions ;
- amélioration de la couverture des tests ;
- intégration possible dans une chaîne CI/CD ;
- génération automatique des résultats.

Dans notre projet, l'automatisation est particulièrement utile car le parcours d'inscription et de commande contient plusieurs étapes qui doivent être vérifiées de manière répétitive.

1.4 Utilisation du Page Object Model

Nous avons choisi d'utiliser le modèle de conception **Page Object Model**, également appelé POM.

Le POM consiste à représenter les différentes pages de l'application par des objets contenant les éléments et les actions associés à ces pages.

Cette approche permet de séparer la logique d'interaction avec l'interface du scénario de test.

Ainsi, le fichier de test décrit principalement le parcours fonctionnel, tandis que les Page Objects regroupent les actions réalisables sur les différentes pages.

Cette organisation permet d'améliorer :

- la lisibilité ;
- la réutilisabilité ;
- la maintenabilité ;
- la séparation des responsabilités ;
- la centralisation des sélecteurs ;
- la réduction de la duplication du code.

1.5 Organisation des Page Objects

Le projet a été organisé autour de plusieurs Page Objects.

Afin de différencier notre projet des autres implémentations, nous avons adapté les noms des pages et des fichiers.

Les principaux éléments sont les suivants :

- **pageHome** : gestion de la page d'accueil et accès à l'inscription ;
- **Achat** : gestion de l'accès à la partie achat ;
- **shop** : gestion de la sélection du produit ;
- **information** : gestion des informations nécessaires à la commande ;
- **byCard** : gestion de la validation du panier et de l'accès au paiement ;
- **testing.cy.js** : fichier principal contenant le scénario de test.

Cette organisation permet de mieux structurer le projet et de faciliter sa maintenance.

1.6 Méthodes utilisées dans le POM

Les Page Objects regroupent principalement deux catégories d'informations : les éléments de la page et les méthodes d'action.

Les éléments permettent d'identifier les composants de l'interface, comme les champs, les liens, les boutons et les cases à cocher.

Les méthodes permettent ensuite d'effectuer les actions associées.

Les principales opérations utilisées sont :

- cliquer sur un élément ;
- saisir une valeur dans un champ ;
- sélectionner une case à cocher ;
- valider un formulaire ;
- accéder à une nouvelle page ;
- vérifier une URL ;
- effectuer une action sur un produit ;
- valider une commande.

L'intérêt est que le test principal peut appeler ces méthodes sans avoir besoin de connaître les détails techniques des sélecteurs.

1.7 Scénario fonctionnel automatisé

Le scénario automatisé représente un parcours utilisateur complet sur le site Glowria.

Le test commence par l'accès à la page d'accueil.

L'utilisateur accède ensuite à son espace compte et démarre le processus d'inscription.

Une adresse électronique est renseignée afin d'accéder à la page d'inscription.

Après la redirection, le formulaire d'inscription est complété avec les différentes informations demandées.

Les données renseignées comprennent notamment :

- le prénom ;
- le nom ;
- l'adresse électronique ;
- la confirmation de l'adresse électronique ;
- le mot de passe ;
- la date de naissance.

Les choix proposés dans le formulaire sont également sélectionnés et les conditions nécessaires à la création du compte sont acceptées.

Le formulaire est ensuite validé.

Après la création du compte, le scénario vérifie que l'utilisateur n'est plus présent sur la page d'inscription.

Le test poursuit ensuite son parcours vers l'espace compte.

L'utilisateur accède à la partie achat puis sélectionne un produit.

Le parcours continue vers la page de commande où les informations nécessaires sont renseignées.

Enfin, la commande est validée et le scénario accède à la plateforme de paiement sécurisée.

1.8 Utilisation de Faker

Afin d'éviter l'utilisation permanente des mêmes données de test, nous avons utilisé la bibliothèque **Faker**.

Faker permet de générer automatiquement des données réalistes.

Dans notre scénario, cette bibliothèque peut notamment générer :

- des adresses électroniques ;
- des prénoms ;
- des noms ;
- des numéros de téléphone.

L'utilisation de données dynamiques présente plusieurs avantages.

Elle permet notamment de réduire les conflits avec des comptes existants et de rendre le scénario plus réutilisable.

Elle permet également de varier les données utilisées lors des différentes exécutions du test.

1.9 Synchronisation avec les requêtes réseau

Lors de l'automatisation, il est important de synchroniser les actions du test avec le comportement réel de l'application.

Une attente fixe, comme une attente de plusieurs secondes, n'est pas toujours adaptée.

Par exemple, une attente fixe peut ralentir inutilement le test si la requête est terminée avant la fin de l'attente.

À l'inverse, si la réponse prend plus de temps que prévu, l'attente peut être insuffisante.

Pour résoudre ce problème, nous avons utilisé le mécanisme d'interception des requêtes réseau proposé par Cypress.

L'interception permet d'attendre une requête précise et donc de synchroniser le test avec un événement réel de l'application.

Cette approche améliore la stabilité du scénario et évite de dépendre d'une durée d'attente arbitraire.

1.10 Configuration de Cypress

Nous avons adapté le fichier de configuration `cypress.config.js` afin de définir une taille de fenêtre adaptée à l'application testée.

La résolution utilisée est :

- largeur : **1280 pixels** ;

— hauteur : **720 pixels**.

Cette configuration a été ajoutée car certains éléments de l'interface ne sont pas toujours visibles lorsque la fenêtre est trop petite.

L'augmentation de la taille de la fenêtre permet d'afficher une plus grande partie de l'interface HTML et facilite les interactions avec les éléments de la page.

Cette configuration est particulièrement utile pour les pages contenant plusieurs éléments de navigation ou des formulaires relativement longs.

1.11 Problèmes rencontrés et solutions

Au cours de la réalisation du projet, plusieurs problèmes ont été rencontrés lors de l'exécution des tests.

Un premier problème concernait certains éléments qui pouvaient être recouverts par d'autres éléments de l'interface.

Dans ce cas, Cypress pouvait empêcher l'interaction avec l'élément car celui-ci n'était pas directement accessible.

Un deuxième problème concernait le bouton de validation du formulaire. Celui-ci pouvait rester désactivé tant que certaines informations obligatoires ou certaines conditions n'étaient pas correctement satisfaites.

La résolution de ce problème a nécessité de respecter l'ordre correct des actions et de vérifier que les champs obligatoires étaient correctement renseignés.

Ces problèmes montrent l'importance de construire un scénario d'automatisation qui reproduit correctement les règles de fonctionnement de l'application.

1.12 Gestion du changement de domaine avec `cy.origin()`

Une étape particulière du scénario concerne le passage du site Glowria vers la plateforme de paiement sécurisée PayZen.

Au cours du parcours, l'utilisateur quitte le domaine principal de Glowria et est redirigé vers le domaine :

`https://secure.payzen.eu`

Il s'agit donc d'un changement d'origine.

Cypress dispose de la commande `cy.origin()` permettant d'exécuter des commandes dans le contexte d'une autre origine.

Dans notre scénario, cette fonctionnalité est utilisée afin de vérifier que l'utilisateur arrive correctement sur la page de paiement.

L'URL finale contient notamment :

`/vads-payment/`

L'utilisation de `cy.origin()` permet donc de gérer correctement cette partie du parcours End-to-End.

Cette étape est importante car elle démontre que le test couvre non seulement le domaine principal de l'application, mais également la redirection vers une plateforme externe utilisée pour le paiement.

1.13 Génération des rapports avec Allure

Afin d'obtenir une représentation claire des résultats des tests, nous avons intégré **Allure**.

Allure est un outil permettant de générer des rapports graphiques à partir des résultats d'exécution des tests.

Le rapport permet notamment de visualiser :

- le nombre de tests exécutés ;
- les tests réussis ;
- les tests échoués ;
- la durée des tests ;
- les suites de tests ;
- les scénarios exécutés ;
- les différentes étapes du test.

L'utilisation d'Allure facilite donc l'analyse des résultats et permet de présenter les résultats de manière professionnelle.

1.14 Résultats de l'automatisation

Les captures suivantes présentent les preuves d'exécution obtenues lors de l'automatisation.

Elles permettent de compléter les résultats techniques par des éléments visuels issus de Cypress et Allure.

1.14.1 Figure 1 – Rapport Allure : vue générale



FIGURE 1.1 – Rapport Allure : vue générale

Résultat de l'exécution avec Allure :

Cette capture présente la vue générale du rapport Allure généré après l'exécution du test automatisé.

Le rapport indique un taux de réussite de **100 %**, avec **1 test case exécuté**.

La suite de tests correspond au parcours fonctionnel d'inscription et de commande.

Cette vue permet d'avoir rapidement une vision globale de l'état de l'exécution et de vérifier que le scénario automatisé s'est terminé avec succès.

1.14.2 Figure 2 – Exécution Cypress et redirection vers PayZen

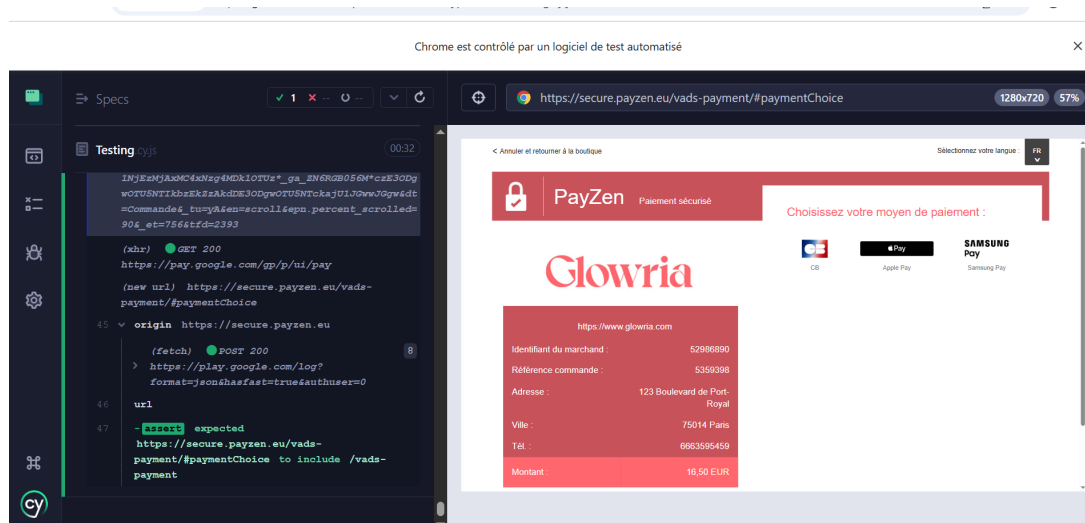


FIGURE 1.2 – Exécution Cypress et redirection vers PayZen

Validation de la redirection vers la plateforme de paiement :

Cette capture montre l'exécution du scénario directement dans le Test Runner Cypress.

Après les différentes étapes du parcours utilisateur sur Glowria, le navigateur est redirigé vers **PayZen**, avec une URL commençant par :

`https://secure.payzen.eu/vads-payment/`

L'assertion affichée dans Cypress est validée, ce qui confirme que la redirection vers la plateforme de paiement a été correctement effectuée.

Cette étape est particulièrement importante car elle implique un changement d'origine entre le domaine Glowria et le domaine sécurisé de PayZen.

Ce changement de domaine est géré dans le test avec le mécanisme `cy.origin()`.

1.14.3 Figure 3 – Rapport Allure : détail de la suite

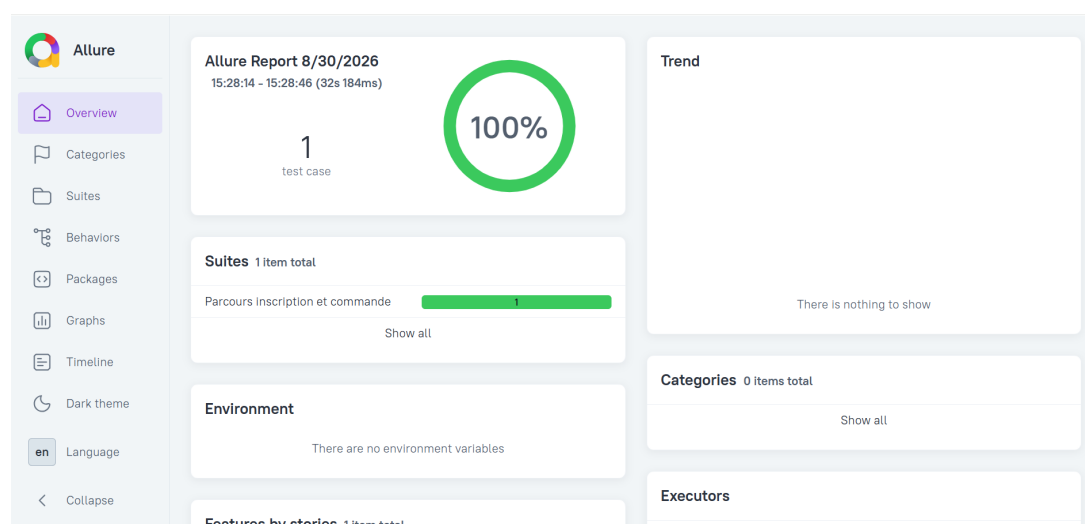


FIGURE 1.3 – Rapport Allure : détail de la suite

Détail de la suite de tests :

Cette capture présente la section **Suites** du rapport Allure.

Elle montre la suite :

« **Parcours inscription et commande** »

ainsi que le test :

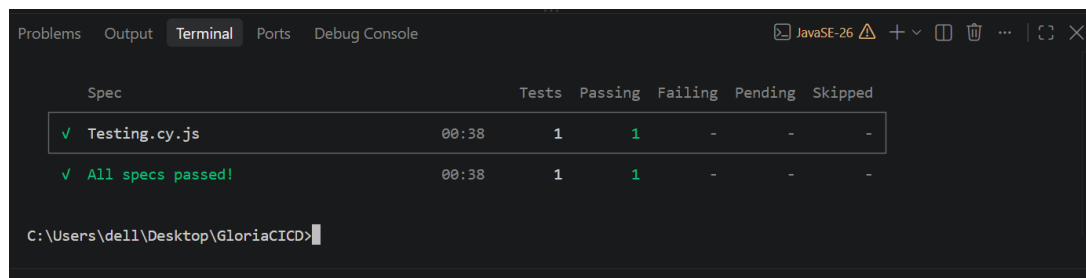
« **Création de compte et commander** »

Le symbole vert indique que le test a été exécuté avec succès.

La durée affichée est de **32 secondes et 184 millisecondes**.

Cette présentation permet d'identifier facilement le scénario exécuté, son statut et sa durée.

1.14.4 Figure 4 – Preuve de réussite du parcours End-to-End



Spec	Tests	Passing	Failing	Pending	Skipped
✓ Testing.cy.js	00:38	1	1	-	-
✓ All specs passed!	00:38	1	1	-	-

C:\Users\dell\Desktop\GloriaCICD>

FIGURE 1.4 – Preuve de réussite du parcours End-to-End

Preuve de réussite du parcours End-to-End :

La capture confirme que le scénario E2E a parcouru les différentes étapes prévues jusqu'à la page de paiement.

Cypress affiche l'exécution des commandes ainsi que l'assertion finale validée sur l'URL PayZen.

Le résultat démontre que l'automatisation ne se limite pas à tester une fonctionnalité isolée.

Elle couvre un parcours utilisateur complet allant de la création du compte jusqu'à l'accès à la plateforme de paiement.

1.14.5 Analyse globale des résultats

Les résultats obtenus montrent que le scénario automatisé a été exécuté avec succès.

Cypress permet de suivre précisément les différentes étapes du parcours utilisateur, tandis qu'Allure fournit une représentation structurée et lisible des résultats.

L'association des deux outils facilite ainsi le suivi, l'analyse et la présentation des tests automatisés.

Le taux de réussite de **100 %** obtenu lors de l'exécution présentée constitue un indicateur positif concernant le bon fonctionnement du scénario automatisé.

Les captures montrent également que le scénario atteint correctement la plateforme de paiement sécurisée, ce qui confirme le bon fonctionnement de la chaîne fonctionnelle testée.

1.15 Intégration continue avec Jenkins

Afin d'intégrer les tests automatisés dans une démarche d'**Intégration Continue et de CI/CD**, nous avons prévu l'utilisation de Jenkins.

Jenkins est un outil d'automatisation permettant d'exécuter automatiquement différentes tâches du projet.

Dans notre contexte, Jenkins peut être utilisé pour automatiser l'exécution des tests Cypress.

Le pipeline peut notamment effectuer les opérations suivantes :

1. récupérer le projet depuis le dépôt Git ;
2. installer les dépendances nécessaires ;
3. préparer l'environnement d'exécution ;
4. lancer les tests Cypress ;
5. récupérer les résultats des tests ;
6. générer les rapports Allure ;
7. signaler la réussite ou l'échec du pipeline.

L'intégration continue permet ainsi d'exécuter automatiquement les tests après une modification du projet.

Cette approche facilite la détection rapide des régressions et améliore la fiabilité du processus de développement.

1.16 Utilisation de Docker

Docker est utilisé afin de disposer d'un environnement d'exécution isolé et reproductible.

L'utilisation d'une image Docker adaptée à Cypress permet de regrouper les éléments nécessaires à l'exécution des tests dans un environnement contrôlé.

Cette approche présente plusieurs avantages :

- environnement reproductible ;
- isolation des dépendances ;
- simplification de l'exécution des tests ;
- compatibilité avec les environnements CI/CD ;
- intégration facilitée avec Jenkins.

Dans une chaîne CI/CD, Jenkins peut ainsi lancer les tests Cypress dans un environnement Docker afin de garantir une exécution plus stable et homogène.

1.16.1 Configuration de Docker avec Jenkins sous Windows

Pour permettre à Jenkins d'utiliser Docker, nous avons mis en place une architecture composée de deux conteneurs principaux.

Le premier conteneur correspond au service Docker-in-Docker (**Docker-in-Docker** ou **DinD**). Il fournit un moteur Docker accessible par Jenkins.

Le deuxième conteneur correspond à Jenkins. Celui-ci contient l'environnement Jenkins ainsi que le client Docker nécessaire pour communiquer avec le moteur Docker.

Les deux conteneurs sont connectés au même réseau Docker nommé `jenkins`.

Cette configuration permet à Jenkins d'utiliser le service Docker accessible avec le nom d'hôte `docker`.

Étape 1 – Création du réseau Docker

La première étape consiste à créer un réseau Docker dédié à Jenkins.

La commande utilisée est :

```
docker network create jenkins
```

Ce réseau permet aux différents conteneurs utilisés par Jenkins de communiquer entre eux.

Étape 2 – Lancement du conteneur Docker-in-Docker

La deuxième étape consiste à lancer un conteneur **Docker-in-Docker**.

Sous Windows avec l'invite de commandes, la commande utilisée est :

```
docker run --name jenkins-docker --rm --detach ^
--privileged --network jenkins --network-alias docker ^
--env DOCKER_TLS_CERTDIR=/certs ^
--volume jenkins-docker-certs:/certs/client ^
--volume jenkins-data:/var/jenkins_home ^
--publish 2376:2376 ^
docker:dind
```

Le paramètre `-privileged` permet au conteneur DinD de fonctionner avec les privilèges nécessaires.

L'option `-network jenkins` connecte le conteneur au réseau créé précédemment.

L'option `-network-alias docker` permet aux autres conteneurs du même réseau d'utiliser le nom `docker` pour accéder au service Docker.

La variable `DOCKER_TLS_CERTDIR` et le volume `jenkins-docker-certs` permettent de mettre en place la communication sécurisée entre Jenkins et Docker.

Étape 3 – Création du Dockerfile Jenkins

La troisième étape consiste à créer un fichier nommé **Dockerfile** permettant de construire une image Jenkins personnalisée.

Le contenu utilisé est le suivant :

```
FROM jenkins/jenkins:2.568.2-jdk21

USER root

RUN apt-get update && apt-get install -y lsb-release

RUN curl -fsSL /usr/share/keyrings/docker-archive-keyring.asc \
    https://download.docker.com/linux/debian/gpg

RUN echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/usr/share/keyrings/docker-archive-keyring.asc] \
    https://download.docker.com/linux/debian \
    $(lsb_release -cs) stable" \
    > /etc/apt/sources.list.d/docker.list

RUN apt-get update && apt-get install -y docker-ce-cli

USER jenkins

RUN jenkins-plugin-cli --plugins \
    "blueocean docker-workflow json-path-api"
```

Cette image contient Jenkins ainsi que le client Docker permettant à Jenkins de communiquer avec le moteur Docker.

Étape 4 – Construction de l'image Jenkins

Une fois le `Dockerfile` créé, l'image Jenkins personnalisée est construite avec la commande :

```
docker build -t myjenkins-blueocean:2.568.2-1 .
```

L'option `-t` permet de donner un nom et une version à l'image créée.

Dans notre cas, l'image est appelée :

```
myjenkins-blueocean:2.568.2-1
```

Cette image peut ensuite être utilisée pour démarrer le conteneur Jenkins.

Étape 5 – Démarrage de Jenkins connecté à Docker

La dernière étape consiste à démarrer le conteneur Jenkins et à le connecter au même réseau que le conteneur Docker-in-Docker.

La commande utilisée est :

```
docker run --name jenkins-blueocean --restart=on-failure --detach ^
--network jenkins ^
--env DOCKER_HOST=tcp://docker:2376 ^
--env DOCKER_CERT_PATH=/certs/client ^
--env DOCKER_TLS_VERIFY=1 ^
--volume jenkins-data:/var/jenkins_home ^
--volume jenkins-docker-certs:/certs/client:ro ^
--publish 8080:8080 ^
--publish 50000:50000 ^
myjenkins-blueocean:2.568.2-1
```

La variable :

```
DOCKER_HOST=tcp://docker:2376
```

indique à Jenkins l'adresse du moteur Docker.

Le nom `docker` correspond à l'alias réseau défini lors du lancement du conteneur Docker-in-Docker.

La variable :

```
DOCKER_CERT_PATH=/certs/client
```

indique l'emplacement des certificats utilisés pour la communication sécurisée.

Enfin :

```
DOCKER_TLS_VERIFY=1
```

active la vérification TLS pour la connexion au moteur Docker.

Le port 8080 permet d'accéder à l'interface Jenkins depuis le navigateur, tandis que le port 50000 est utilisé pour la communication avec les agents Jenkins lorsque cela est nécessaire.

1.16.2 Architecture de communication Docker et Jenkins

L'architecture mise en place peut être résumée de la manière suivante :

Composant	Fonction
Réseau jenkins	Permet la communication entre les conteneurs
jenkins-docker	Fournit le moteur Docker-in-Docker
Alias docker	Nom utilisé par Jenkins pour accéder au moteur Docker
jenkins-blueocean	Conteneur contenant Jenkins et le client Docker
Port 2376	Communication Docker sécurisée avec TLS
Port 8080	Accès à l'interface Web Jenkins
Port 50000	Communication Jenkins avec les agents

Cette architecture permet donc à Jenkins d'exécuter des commandes Docker depuis son propre conteneur.

1.16.3 Problème de communication entre Jenkins et Docker

Lors de la première exécution du pipeline Jenkins, une erreur de connexion au moteur Docker a été rencontrée.

Le message indiquait notamment :

```
dial tcp: lookup docker on 127.0.0.11:53: no such host
```

Cette erreur signifie que Jenkins ne pouvait pas résoudre le nom d'hôte **docker**.

Le pipeline essayait pourtant d'utiliser :

```
DOCKER_HOST=tcp://docker:2376
```

La cause était donc liée à la communication entre les conteneurs.

Pour résoudre ce problème, il est nécessaire que Jenkins et Docker-in-Docker soient connectés au même réseau Docker et que le conteneur Docker-in-Docker possède l'alias réseau :

```
docker
```

La configuration avec :

```
--network jenkins
--network-alias docker
```

permet à Jenkins de résoudre le nom **docker** et d'établir la connexion avec le moteur Docker.

Cette configuration constitue donc une étape importante pour l'intégration de Jenkins avec Docker.

1.16.4 Intégration de Docker dans le pipeline Jenkins

Une fois Jenkins connecté à Docker, le pipeline peut utiliser une image Cypress adaptée à la version du projet.

Dans notre projet, l'image utilisée pour l'exécution des tests est :

```
cypress/included:14.5.4
```

Cette image permet d'exécuter Cypress dans un environnement préparé contenant les composants nécessaires.

Le pipeline peut ensuite installer les dépendances avec :

```
npm ci
```

Puis lancer les tests avec :

```
npx cypress run
```

Lorsque l'option Allure est activée, les résultats peuvent être générés puis archivés dans Jenkins.

Cette organisation permet de réaliser automatiquement la chaîne suivante :

Git → Jenkins → Docker → Cypress → Allure

1.16.5 Figure 5 – Résultat de l'exécution du pipeline Jenkins



FIGURE 1.5 – Résultat de l'exécution du pipeline Jenkins

Résultat de l'intégration continue :

Cette capture présente le résultat de l'exécution du pipeline Jenkins après la mise en place de la communication entre Jenkins et Docker.

Elle constitue une preuve visuelle de l'exécution du processus d'intégration continue.

Le pipeline permet de récupérer le projet, de préparer l'environnement d'exécution, de lancer les tests automatisés et de récupérer les résultats.

La réussite du pipeline confirme que l'environnement Jenkins est correctement configuré pour exécuter le projet automatisé.

L'utilisation de Docker permet également de conserver un environnement d'exécution homogène entre les différentes exécutions du pipeline.

Cette intégration constitue une étape importante vers une chaîne CI/CD complète pour l'automatisation des tests.

1.17 Architecture globale de la solution

L'organisation générale de la solution peut être résumée par les différents composants utilisés dans le projet.

Élément	Rôle
Cypress	Automatisation des tests End-to-End
POM	Organisation et réutilisation des actions des pages
Faker	Génération de données de test dynamiques
intercept	Synchronisation avec les requêtes réseau
cy.origin()	Gestion du changement de domaine
Allure	Génération des rapports de tests
Jenkins	Automatisation du pipeline CI/CD
Docker	Création d'un environnement reproductible
Git	Gestion et versionnement du projet

La combinaison de ces différents outils permet d'obtenir une solution complète d'automatisation des tests.

1.18 Bilan du projet

La réalisation de ce projet nous a permis de mettre en place un scénario complet d'automatisation d'un parcours utilisateur.

Cypress a permis d'automatiser les interactions avec le navigateur et de vérifier automatiquement les résultats attendus.

Le choix du Page Object Model a permis d'améliorer l'organisation du projet en séparant les actions liées aux différentes pages du scénario de test.

L'utilisation de Faker permet de gérer les données de test de manière dynamique et d'éviter la dépendance à des données fixes.

Le mécanisme d'interception des requêtes réseau améliore la synchronisation du scénario et évite l'utilisation excessive d'attentes fixes.

La commande `cy.origin()` permet quant à elle de gérer le changement de domaine vers la plateforme de paiement PayZen.

Allure apporte une représentation graphique des résultats et permet de suivre facilement le statut et la durée des tests.

Les résultats présentés montrent un taux de réussite de **100 %** pour le scénario exécuté.

Enfin, Jenkins et Docker permettent d'envisager l'intégration des tests dans une chaîne CI/CD afin d'automatiser leur exécution dans un environnement contrôlé.

1.19 Conclusion

Ce projet montre l'intérêt de l'automatisation des tests dans le cadre d'une application web.

La combinaison de Cypress et du Page Object Model permet d'obtenir des tests structurés, lisibles, réutilisables et plus faciles à maintenir.

L'utilisation de données dynamiques avec Faker améliore la flexibilité des tests.

Le mécanisme `intercept` permet de synchroniser les tests avec les requêtes réelles de l'application, tandis que `cy.origin()` permet de gérer le changement d'origine lors de la redirection vers la plateforme de paiement.

La génération de rapports avec Allure apporte une meilleure visibilité sur les résultats obtenus.

Les captures présentées dans ce rapport montrent que le scénario automatisé a été exécuté avec succès et qu'il atteint correctement la plateforme de paiement sécurisée.

Enfin, l'intégration de Jenkins et Docker permet d'envisager une chaîne CI/CD complète dans laquelle les tests peuvent être exécutés automatiquement après chaque modification du projet.

Cette organisation constitue donc une base solide pour mettre en place une stratégie d'automatisation des tests et contribuer à l'amélioration continue de la qualité logicielle.